

Chapter 4: Dynamic Programming

*Lecturer: Jasper Lee**Scribe: Jasper Lee*

This chapter introduces the powerful algorithmic design paradigm of dynamic programming, which stems from the simple-sounding observation that repeated computation can be optimized away by storing the results.

1 Introduction to Dynamic Programming

Let us use recursion as a starting point. Generically, recursion breaks down a larger problem into subproblems of the same kind, and recurses until the computation reaches base cases.

In prior Divide and Conquer algorithms we saw, the subproblems never overlap with each other. Every recursive call is handling a different subproblem or a different part of the input. On the other hand, the art of Dynamic Programming is to structure the recursion in a way such that the subproblems can overlap, and the key to the computation is to store the subproblem results so they can be reused efficiently without re-computation.

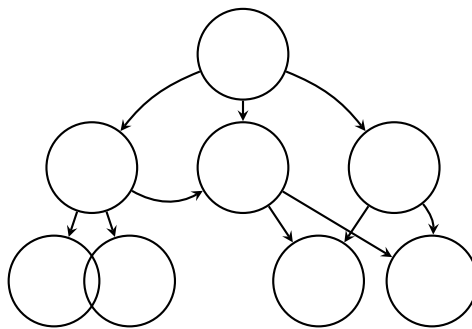


Figure 1: A recursion tree with overlapping subproblems. This is formally a directed acyclic graph.

As a simple but informative example, consider the Fibonacci sequence, defined by the recurrence relation $F(n+2) = F(n+1) + F(n)$, with base cases $F(0) = 1$ and $F(1) = 1$. If we implement this definition directly as a recursive function, the runtime grows proportionally to the Fibonacci numbers themselves, resulting in an exponential $\Theta(F(n))$ time complexity.

Instead of running an exponential time recursion, we can in fact just store the Fibonacci numbers in an array, and compute them using a simple for loop. This is a very simple instance of a dynamic programming algorithm.

Algorithm 1 Bottom-Up Fibonacci

1. Initialize array $F \leftarrow$ zeros of size $n + 1$
 2. $F[0] \leftarrow 0$; $F[1] \leftarrow 1$
 3. for $i = 2$ to n :
 - (a) $F[i] \leftarrow F[i - 1] + F[i - 2]$
 4. return $F[n]$
-

This algorithm runs in $O(n)$ time and $O(n)$ space. In fact, we can further optimize the space complexity to $O(1)$ by observing that we only ever need the two most recently computed values.

1.1 Components of a (Standard) DP Algorithm

When designing a standard Dynamic Programming algorithm, there are four core components we must precisely specify:

1. **Table entries:** Dynamic programming stores computation results, which we can generically consider as a table, where each entry can be considered a subproblem of the original problem (after some transformation). To specify the table, we have to state the precise meaning of the table entries and how the meaning depends on the indices of the entries.
2. **Recurrence:** How are the table entries mathematically related to one another?
3. **(Pseudo)code to fill the table:** There are typically two ways (*bottom-up* vs *top-down*) to actually implement a dynamic programming algorithm, once the table and the recurrence are specified. The Fibonacci implementation above is bottom-up via iteration, and right below we will see a top-down implementation via recursion.
4. **Wrap Up:** How can the final answer to the original problem be computed from the fully-filled table?

Here we give a top-down implementation for the Fibonacci numbers. Essentially, we just implement the recurrence relation directly via recursion, and store the subproblem results in a global table. At the beginning of each recursive call, we check whether the subproblem has already been computed; if yes, simply return the table entry, otherwise we proceed with the recursive computation.

Algorithm 2 Top-Down Fibonacci

Global variable: array $F[0 \dots n]$ initialized to -1

function FIB(n)

1. if $F[n] \neq -1$: return $F[n]$
 2. if $n = 0$: $F[0] \leftarrow 0$; return 0
 3. if $n = 1$: $F[1] \leftarrow 1$; return 1
 4. $F[n] \leftarrow \text{FIB}(n - 1) + \text{FIB}(n - 2)$
 5. return $F[n]$
-

2 Counting Paths in a Grid

Consider an $m \times n$ grid where some cells are blocked, and we wish to count the total number of valid paths from the start cell $(1, 1)$ to the end cell (m, n) . The allowable moves are strictly restricted to moving right or moving down, namely, moving from (i, j) to $(i+1, j)$ or $(i, j+1)$.

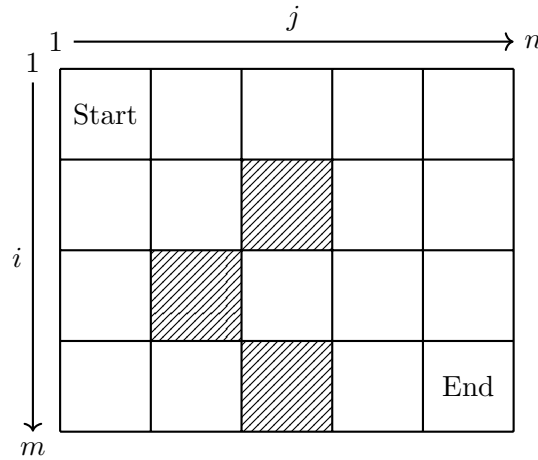


Figure 2: An $m \times n$ grid containing explicitly blocked cells. The goal is to traverse from the top-left to the bottom-right using only valid down and right moves.

To apply Dynamic Programming, we specify the four components.

- **Table Entries:** What are possible table structures and precise table entry meaning? Perhaps the obvious thing to try is to follow the grid structure, with a table indexed by (i, j) . In terms of the *meaning* of the table entry, we can try directly generalizing the overall problem: Let $T(i, j)$ be the number of valid paths from $(1, 1)$ to (i, j) .
- **Recurrence:** How do these entries relate? Any valid path reaching (i, j) **must** make its final step from either the cell directly above, $(i - 1, j)$, or the cell directly to the

left, $(i, j - 1)$ ¹. The numbers of such paths are $T(i - 1, j)$ and $T(i, j - 1)$ respectively. These two sets of paths are also mutually exclusive, so we can conclude:

$$T(i, j) = T(i - 1, j) + T(i, j - 1)$$

for non-blocked cells. Our base case is $T(1, 1) = 1$. For any cell that is blocked or have out-of-bounds indices (where $i = 0$ or $j = 0$), $T(i, j) = 0$.

• **Pseudocode:**

Algorithm 3 Count Grid Paths

1. Initialize table $T[0..m, 0..n]$ with zeros.
 2. $T[1, 1] \leftarrow 1$
 3. for $i = 1$ to m :
 - (a) for $j = 1$ to n :
 - i. if $(i, j) = (1, 1)$, continue
 - ii. if (i, j) is blocked, $T[i, j] \leftarrow 0$
 - iii. else $T[i, j] \leftarrow T[i - 1, j] + T[i, j - 1]$
 4. return $T[m, n]$
-

Runtime: The nested loops clearly take $O(mn)$ time. There are $O(mn)$ table entries, and each entry takes $O(1)$ computation time to compute the recurrence.

Correctness: We already showed that the DP table recurrence is true. The correctness of the computation follows from an implicit induction argument: the order of our double for-loop ensures that by the time we compute $T[i, j]$, the subproblems $T[i - 1, j]$ and $T[i, j - 1]$ have already been computed. This type of sentence is all you need to say for homeworks and exams.

3 Maximum Subarray Sum

Suppose we are given an array $A[1..n]$, and we wish to compute the maximum sum of any contiguous subarray, denoted mathematically as $\max_{i,j} \sum(A[i..j])$. Note that the chosen i can be strictly greater than j , denoting an empty sum. This can happen when the entire array consists of negative numbers.

A naïve algorithm tries all possible pairs of subarray boundaries (i, j) and computes the sum for each subarray, yielding $O(n^2)$ runtime. Our goal is to improve this to $O(n)$ time using dynamic programming.

Sometimes, the target runtime of the dynamic programming algorithm can give hints as to what the table can look like. Since we restrict ourselves to a $O(n)$ time algorithm,

¹This is like the kind of case analysis we did for Divide and Conquer, making a precise claim and argument about covering all cases of what solutions look like.

the DP table can only be $O(n)$ in size, as each entry must take at least some unit time to process.

Given that, for our first try, let us consider a table structure $T_{\max}[i]$ for $i \in [1..n]$. What is a reasonable table entry definition for this single row DP table? We can, again, try directly generalizing the overall problem, by *restricting the input* considered. Specifically, let us define

$$T[i] = \text{the maximum subarray sum of the prefix } A[1..i].$$

Given this DP table definition, we can now try deriving a corresponding recurrence. To do so, we again need to make an observation on the structure of an optimal solution, covering all possible cases. Here, we observe that any optimal solution to $T_{\max}[i]$ **must** either use $A[i]$ in the sum, or it does not. In the case it does not, the optimal solution must also be optimal for the subproblem of $T[i - 1]$. However, if it does, then we are left with many possibilities still. All we know is that the desired solution does use $A[i]$, so we have to consider $A[i]$, $A[(i - 1)..i]$, $A[(i - 2)..i]$ all the way to $A[1..i]$. This gives a long recurrence

$$\begin{aligned} T_{\max}[i] = \max(&T_{\max}[i - 1], \text{sum}(A[i]), \\ &\text{sum}(A[(i - 1)..i]), \\ &\text{sum}(A[(i - 2)..i]), \\ &\vdots \\ &\text{sum}(A[1..i])) \end{aligned}$$

This is indeed a dynamic programming algorithm, but if we had implemented this, the runtime would be $\Omega(n^2)$ since table entry $T_{\max}[i]$ requires at least $\Omega(i)$ computation, even if the sums can each be evaluated instantly in unit time (which they can be if the algorithm is appropriately structured).

Our next observation is that, the bulk of the work inside the $T[i]$ recurrence is actually solving a slightly different algorithmic problem, which itself can be tackled by dynamic programming! Specifically, we needed to compute the maximum subarray sum of $A[1..i]$ that *has* to use $A[i]$. Define this to be $T_{\text{suffix}}[i]$. The structural observation for the recurrence of $T_{\text{suffix}}[i]$ is similar to the one for $T_{\max}[i]$, that any such optimal solution must either use $A[i - 1]$ or not. In the “not” case, the subarray is only $A[i]$. In the “yes” case, any optimal solution for $T_{\text{suffix}}[i]$ that uses $A[i - 1]$ might as well use an optimal solution for $T_{\text{suffix}}[i - 1]$, yielding a value of $T_{\text{suffix}}[i - 1] + A[i]$. Summarizing, $T_{\text{suffix}}[i - 1]$ has the recurrence

$$T_{\text{suffix}}[i] = \max(A[i], T_{\text{suffix}}[i - 1] + A[i])$$

Now it is straightforward to compute T_{suffix} in $O(n)$ time. With T_{suffix} computed, we can write down a new recurrence for the original $T_{\max}$ table as

$$T_{\max}[i] = \max(T_{\max}[i - 1], T_{\text{suffix}}[i])$$

which of course now takes only $O(n)$ time to compute as well.

4 Answering Range Queries with Prefix Sums

Here is a simple but incredibly useful trick from doing dynamic programming.

Setting: Given an array $A[1 \dots n]$, we wish to prepare a data structure to answer range queries $(i, j) \mapsto \sum_{k=i}^j A[k]$.

A naïve algorithm just evaluates each query directly, using $O(j - i)$ time per query.

We can improve this significantly by observing that contiguous sums can be expressed as the difference between *prefix sums*:

$$\sum_{k=i}^j A[k] = \sum_{k=1}^j A[k] - \sum_{k=1}^{i-1} A[k]$$

We can precompute the prefix sums of A in $O(n)$ time.

- **Table Entries:** $T[i] = \sum_{k=1}^i A[k]$
- **Recurrence:** $T[i + 1] = T[i] + A[i + 1]$

After this $O(n)$ preprocessing step, any arbitrary range query (i, j) can be answered in $O(1)$ time simply by evaluating $T[j] - T[i - 1]$.

5 Longest Common Subsequence

Setting: Given a sequence S , a subsequence is formed by deleting some symbols from a sequence without changing the relative order of the remaining symbols. For example, if $S = \text{"abcdefg"}$, then "adf" , "adfg" , "bce" are all valid subsequences. Generally, for a sequence of length n , there are always 2^n many subsequences (including the empty one).

Given two sequences $A[1 \dots m]$ and $B[1 \dots n]$, our goal is to find the length of the longest common subsequence (LCS) appearing in both A and B .

Naïve exhaustive search would take time proportional to $\exp(m) \exp(n)$. Instead, we will derive an algorithm that runs in $O(mn)$ time. As an aside, a recently-developed research subfield of fine-grained complexity conjectures that there is no $O(m^{.999} n^{.999})$ time algorithm. The LCS is known to be “SETH-hard” problem, where SETH stands for Strong Exponential Time Hypothesis: a hypothesis that is stronger than $P \neq NP$ but which is also believed by many researchers.

Given our target runtime, we can afford a table of size $O(mn)$. We will try the approach from previous examples, which is to define table entries as the LCS problem itself, but on a restricted subset of the input.

- **Table Entries:** Let $T[i, j]$ be the length of the LCS for the prefixes $A[1..i]$ and $B[1..j]$.
- **Recurrence:** Any longest common subsequence must either include the characters $A[i]$ and $B[j]$ (if they are equal symbols), or it does not use $A[i]$, or it does not use $B[j]$. Taking the maximum among these exhaustive cases yields the recurrence:

$$T[i, j] = \max \begin{cases} 1 + T[i - 1, j - 1] & \text{if } A[i] = B[j] \\ T[i - 1, j] \\ T[i, j - 1] \end{cases}$$

(Note: If $A[i] = B[j]$, it is always strictly optimal to match them, but expressing the recurrence as a maximum over all branches is safe and easier to prove correct, as we did above.)

- **Pseudocode:** The DP table can be filled using a double loop, just like in the [Section 2](#) example.
- **Wrap Up:** The final answer is found at $T[m, n]$.

Exercise: Design an $O(mn)$ time algorithm to find the longest common *substring*, where a substring is a *contiguous* subsequence.

6 0-1 Knapsack

Setting: We have a knapsack with an integer capacity C . We are given a list of n items, where each item i has a reward r_i and an integer weight w_i . We want to find a subset of items such that their total weight is at most C and their total reward is maximized. The “0-1” in the problem name means that we can either take an item or not; there is no way to take an item fractionally, nor is there a way to take multiple copies of an item.

This is one of many different formulations of the knapsack problems in the wild. For example, other formulations can assume integer rewards instead of integer item weights/knapsack capacity. Different formulations will require different algorithms, even if they share some conceptual commonalities.

We have not really discussed greedy algorithms yet, but would being greedy work for this problem? No!

For example, a natural greedy algorithm would be to choose items in descending order of r_i/w_i , namely the “reward density”. But here is a simple bad example: capacity $C = 10$, item 1 has $r_1 = 1, w_1 = 10^{-5}$, item 2 has $r_2 = 10, w_2 = 10$.

We will instead solve the problem using dynamic programming.

Our first attempt is to replicate the strategy before, defining a DP table based on the same problem but on restricted inputs:

$$T_0[i] = \text{best 0-1 knapsack solution on items } 1..i$$

But unfortunately this essentially cannot work, because the capacity constraint is not included in the problem definition, and it is insufficient for all the subproblems to have the same capacity C .

So in our next attempt, we instead try a 2-dimensional DP table.

- **Table Entries:** Let $T[i, C]$ be the maximum reward for 0-1 knapsack problem on items $1..i$ under a maximum capacity C . Here, the “remaining” capacity of the knapsack is information that requires tracking, and we use a table index to do so. This is a general technique that is frequently (but not always) useful.
- **Recurrence:** An optimal subset of items either uses item i , or it does not. If it does not, then the subproblem is simply restricted to items $1..(i-1)$ with the same capacity C , corresponding to table entry $T[i-1, C]$. If it does use item i , then after excluding item i , this reduces to a 0-1 knapsack instance on items $1..(i-1)$ but now with capacity $C - w_i$. This reasoning yields the recurrence:

$$T[i, C] = \max(r_i + T[i-1, C - w_i], T[i-1, C])$$

Here, $T[i, C]$ is defined to be $-\infty$ if $C < 0$.

- **Computation:** A double for-loop, for example, with i as outer loop and C as inner loop (or vice versa).
- **Wrap Up:** The solution to our original problem, by definition of our subproblem formulation, resides at $T[n, C]$.

This algorithm takes $O(Cn)$ time, and also $O(Cn)$ space to store the entire 2-dimensional DP table. But we can actually do better on the space front. Observe that, in order to compute the row $T[i, \cdot]$, we only need to row $T[i - 1, \cdot]$. By keeping only one row, we can optimize the memory usage to $O(C)$ space only.

7 Dynamic Programming on Trees

Dynamic programming can apply beyond the rectangular table structure in the previous examples. Another common DP table structure is to follow an input tree.

We have actually already seen instances of this when studying Divide and Conquer algorithms, in the Longest Monochromatic Path example recapped below. Generally, dynamic programming on trees does frequently incorporate elements of Divide and Conquer, given the natural recursive structure of trees.

7.1 Longest Monochromatic Path

We revisit the problem of finding the longest monochromatic (all-red) path in a binary tree, formalizing it through the lens of dynamic programming.

DP Table: We define two state variables for each node:

- $Len[node]$: the length of the longest all-red path strictly within the subtree rooted at $node$.
- $Len_root[node]$: the length of the longest all-red path that starts at $node$ and lies entirely within the subtree rooted at $node$.

Recurrence:

$$Len_root[node] = \begin{cases} 1 + \max(Len_root[node.left], Len_root[node.right]) & \text{if } node \text{ is red} \\ 0 & \text{otherwise} \end{cases}$$

$$Len[node] = \max \left(\begin{aligned} &Len[node.left], Len[node.right], \\ &\begin{cases} 1 + Len_root[node.left] + Len_root[node.right] & \text{if } node \text{ is red} \\ 0 & \text{otherwise} \end{cases} \end{aligned} \right)$$

The recurrence can even be generalized to trees that are not binary, which is left as an exercise. For each DP table entry (namely, for each node), the recurrence should be computable in $O(\# \text{ number of children})$ time. Then, summing over all nodes, the total runtime will be $O(\text{tree size})$ which is linear time.

7.2 Maximum Independent Set on Binary Trees

Setting: Given a binary tree T , the goal is to choose the maximum number of vertices such that no edge has both of its endpoints chosen.

What is the core “either/or” or “cover all cases” observation here? For this problem, it is again a simple observation: for any vertex in the tree, an independent set will either include that vertex, or it will not.

We will again use a DP table that follows the input tree structure instead of a rectangular table.

DP Table: Let us try the obvious table definition:

$A[node]$ = the size of the maximum independent set in the subtree rooted at $node$

How do we write a recurrence for this DP table? If a solution does not take $node$, then it might as well take best solutions for its children, the union of which is a valid independent set. If it does take $node$, it cannot take any of $node$ ’s children, but it could take the grandchildren, so it might as well take the best independent sets for each grandchildren subtree, whose union with $\{node\}$, again, is indeed a valid independent set.

Recurrence:

$$A[node] = \max \begin{cases} A[node.left] + A[node.right] & \text{(not taking } node) \\ 1 + A[node.left.left] \\ \quad + A[node.left.right] \\ \quad + A[node.right.left] \\ \quad + A[node.right.right] & \text{(taking } node) \end{cases}$$

Runtime The work per node is $O(\# \text{ children} + \# \text{ grandchildren})$. Summing over the tree yields $O(\text{tree size})$ total work.

While the first attempt works, there is in fact an alternative cleaner dynamic programming approach that avoids looking at grandchildren, using a slightly augmented table structure.

DP Table: Let $A[node, can_take]$ where can_take is a Boolean, defined as

- If $can_take = 1$, it stores the max independent set size for the subtree rooted at $node$.
- If $can_take = 0$, it stores the max independent set size for the subtree rooted at $node$ with the restriction that $node$ **cannot** be included.

Recurrences: The above DP table definition yields a much cleaner recurrence.

$$\begin{aligned} A[node, 0] &= A[node.left, 1] + A[node.right, 1] \\ A[node, 1] &= \max(A[node, 0], 1 + A[node.left, 0] + A[node.right, 0]) \end{aligned}$$

The correctness analysis is largely the same as the previous algorithm, but this formulation requires only looking at immediate children.

Exercise: Can you generalize both dynamic programming approaches to trees that are not binary? Again, the goal is that the recurrence should only induce work proportional to the number of children (or grandchildren), so that summing over the tree, the total amount of work is linear in the tree size.

7.3 Maximum Matching in Binary Trees

Definition 4.1. A matching (in a graph) is a subset of edges such that each vertex is incident to at most one edge in the matching.

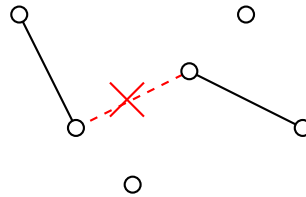


Figure 3: Visualizing the vertex constraint of a matching. Solid black edges form a valid matching because they share no vertices. The dashed red edge cannot be further included because it would result in a vertex being incident to more than one edge.

In this section, our algorithm task is to compute the maximum size across all matchings, in $O(n)$ time for a binary tree.

The key structural “either or” observation is that for any fixed edge, a matching either takes the edge into the matching or it does not.

DP Table: We can define a DP table similarly to our second approach for Maximum Independent Set on trees. The table entry $M[node, can_take]$ is defined as

- $M[node, 1]$: the maximum matching size in the subtree rooted at $node$.
- $M[node, 0]$: the maximum matching size in the subtree rooted at $node$ that does **not** take any edge incident to $node$.

Recurrence:

$$M[node, 0] = M[node.left, 1] + M[node.right, 1]$$

$$M[node, 1] = \max \begin{cases} M[node, 0] & \text{(do not take any edge to a child)} \\ 1 + M[node.left, 0] + M[node.right, 1] & \text{(take edge to left child)} \\ 1 + M[node.right, 0] + M[node.left, 1] & \text{(take edge to right child)} \end{cases}$$

We end this section with a formal proof of correctness for the recurrence.

Correctness

$M[node, 0]$: Every matching not taking an edge from $node$ to a child is a union of matchings in the left and right subtrees. Similarly, any such union of matchings forms a valid matching for the subtree rooted at $node$, without taking an edge incident to $node$. To form an optimal solution, one might as well take the optimal solutions for the left and right children subtrees.

$M[node, 1]$: An optimal matching either takes an edge to a child, or it does not. If it does not, the best solution is just the one for $M[node, 0]$. Otherwise, the matching must take exactly one edge, either to the left child or to the right child. Without loss of generality, say the left edge is taken, then any solution must not take any other edge for the left child. Consequently, in this case, the best solution conditioned on taking the *node-to-left-child* edge is $1 + M[node.left, 0] + M[node.right, 1]$. A symmetric argument argues the right child case, showing the correctness of the recurrence.

8 Shortest Paths on Graphs

Setting: We are given a directed, weighted n -vertex graph where each edge e has weight w_e . These weights may be negative. Our goal is to compute the shortest paths between vertices.

There are two main variations of this problem:

- **Single-Source:** Given a specific vertex u , compute the shortest path from u to all other vertices v .
- **All-Pairs:** For all pairs of vertices u and v , compute the shortest path from u to v .

8.1 Single-Source Shortest Path: Bellman-Ford

Our goal here is to derive the Bellman-Ford algorithm using dynamic programming. This will, interestingly, require different intuition from the basic “either/or” case analysis we have been using in previous examples.

To start, need to define the DP table. We could try the approach we had on trees, and define $D[v]$ (D for distance) as

$$D[v] = \text{shortest path length from } u \text{ to } v$$

The subproblem structure is implicit in what the shortest paths are: every path is a sequence of hops, and a vertex with fewer hops in a shortest path gives a subproblem for vertices with more hops further along the path. But this is circular, since the shortest paths are exactly what we are trying to compute!

Given this hop intuition, we form another DP table that tracks the hop count to define subproblems.

DP Table: Let $D[v, k]$ be the shortest path distance from u to v using at most k hops. We can also insert self-loops of weight 0 for all vertices, and equivalently define $D[v, k]$ be the shortest path distance from u to v using *exactly* k hops.

Recurrence: Defining $w_{x \rightarrow v}$ to be ∞ whenever the edge $x \rightarrow v$ does not exist, the following is the recurrence for the $D[v, k]$ table.

$$D[v, 0] = \begin{cases} \infty & \text{if } v \neq u \\ 0 & \text{if } v = u \end{cases}$$

$$D[v, k] = \min \left(D[v, k-1], \min_{x \neq v} (D[x, k-1] + w_{x \rightarrow v}) \right)$$

We should be careful that defining a table also requires defining the bounds on the table indices. Here, this means answering how large k can be: the maximum number of edges on any shortest path is $|V| - 1$, at least, unless the graph has a negative cycle (a cycle whose edge weights sum to below 0), in which case the notion of shortest paths is not even defined. More on negative cycles later.

Algorithm 4 Bellman-Ford Algorithm

1. For all k , initialize $D[v, k] \leftarrow \infty$ for all $v \neq u$ and $D[u, k] \leftarrow 0$.
 2. for $k = 1$ to $n - 1$:
 - for each edge $x \rightarrow v$:

$$D[v, k] = \min(D[v, k], w_{x \rightarrow v} + D[x, k - 1])$$
-

Implementation: Here, in this implementation, we only enumerate edges present in the graph instead of all pairs (x, v) . Thus, only $O(|E|)$ computation is done in each outer loop, totally to an $O(|V||E|)$ runtime.

We can, again, optimize the space usage of the Bellman-Ford algorithm. As in prior examples, we only need the values at index $k - 1$ to compute the values at index k . In fact, it is possible to drop the hop-count index altogether and just keep updating a single D array, though the correctness analysis is trickier and omitted here.

Algorithm 5 Space-Optimized Bellman-Ford Algorithm

1. Initialize $D[v] \leftarrow \infty$ for all $v \neq u$ and $D[u] \leftarrow 0$.
 2. for $k = 1$ to $n - 1$:
 - for each edge $x \rightarrow v$:

$$D[v] = \min(D[v], w_{x \rightarrow v} + D[x])$$
-

8.2 Negative Cycles

However, as mentioned above, weighted graphs may contain negative cycles, and the notion of shortest paths (absent constraints on the number of hops) might not even be well-defined. It turns out that Bellman-Ford can even be used to detect the existence of negative cycles reachable from the source vertex u , by running the outer loop one more time. To see this, we state and prove the following claim.

Claim 4.2. *There is a negative cycle reachable from u if and only if there exists some vertex v such that $D[v, n] < D[v, n - 1]$.*

Proof. $D[v, n] < D[v, n - 1] \Rightarrow$ existence of negative cycle reachable from u : Let $u \rightarrow a_1 \rightarrow \dots \rightarrow a_i \rightarrow \dots \rightarrow a_j \rightarrow a_n = v$ be an n -hop path of minimum weight for $D[v, n]$.

Now consider the following $n - 1$ hop path $u \rightarrow a_1 \rightarrow \dots \rightarrow a_i \rightarrow a_{j+1} \rightarrow \dots \rightarrow v \rightarrow v \rightarrow \dots \rightarrow v$ where the padding by v makes the path exactly $n - 1$ hops. The weight of this path upper bounds $D[v, n - 1]$, which in turn is strictly greater than $D[v, n]$ that is equal to the weight of the previous path.

The weight of the cycle $a_i \rightarrow \dots \rightarrow a_j = a_i$ is equal to the difference between the weights of the two paths, and is hence strictly negative.

(For all v , $D[v, n] \geq D[v, n-1]$) $\Rightarrow$ there are no negative cycles reachable from u : Consider any arbitrary cycle $a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_1$ that is reachable from u . For any i , we know that

$$D[a_{i+1}, n-1] \leq D[a_{i+1}, n] \leq D[a_i, n-1] + w_{a_i \rightarrow a_{i+1}}$$

Summing this inequality over all edges in the cycle yields:

$$\sum_i D[a_{i+1}, n-1] \leq \sum_i D[a_i, n-1] + \sum_i w_{a_i \rightarrow a_{i+1}}$$

The total weights of the cycle cancel out, from both sides, leaving $\sum_i w_{a_i \rightarrow a_{i+1}} \geq 0$. $\square$

Algorithmically, we can simply run one additional outer loop (the n -th iteration) to check if any distances are updated. If they are, a negative cycle exists.

8.3 All-Pairs Shortest Path

If we simply run Bellman-Ford using every vertex as a source, we can compute all-pairs shortest path in $O(|V|^2|E|)$ time. However, a graph can have as many as $O(|V|^2)$ edges, making this basic approach requiring $O(|V|^4)$ time in the worst case.

The Floyd-Warshall algorithm improves this to $O(|V|^3)$ time. It crucially assumes that there are no negative cycles in the graph.

The subproblem structure here is not necessarily obvious.

DP Table: The DP table has entries $D[i, j, k]$, where i, j and k are all vertices. The entry is defined as

$D[i, j, k]$ = the shortest path length from i to j ,
where all intermediate vertices on the path are constrained to lie in $[1..k]$

Recurrence: To derive the recurrence, the “either or” observation is that a shortest path for $D[i, j, k]$ must either use vertex k exactly once or it does not. If k is not used, then $D[i, j, k-1]$ is the best possible path. If k is used, then the path must go from i to k using vertices in $[1..(k-1)]$ then from k to j using vertices in $[1..(k-1)]$. This yields the recurrence

$$D[i, j, k] = \min(D[i, j, k-1], D[i, k, k-1] + D[k, j, k-1])$$

$$D[i, j, 0] = w_{i \rightarrow j} \quad (\text{which may be } \infty \text{ for non-existent edges})$$

Pseudocode: The algorithm is implemented as three nested loops. Here it is crucial that k is the outer loop, but the nesting order of the inner loops does not matter.

Algorithm 6 Floyd-Warshall Algorithm

for $k = 1$ to n :

 for $i = 1$ to n :

 for $j = 1$ to n :

$$D[i, j, k] \leftarrow \min(D[i, j, k-1], D[i, k, k-1] + D[k, j, k-1])$$

Similar to Bellman-Ford, we can optimize the space complexity by dropping the k index entirely, updating the 2D array in place:

$$D[i, j] = \min(D[i, j], D[i, k] + D[k, j])$$